

Tuần 3: C++ cơ bản, Con trỏ và Quản lý bộ nhớ Stack & Heap / Week 3: C++ Basics, Pointers, and Stack/Heap Memory Management

Mục Tiêu / Objectives

Tiếng Việt (Vietnamese)

- Nắm vững cú pháp cơ bản của ngôn ngữ C++: biến, kiểu dữ liệu, cấu trúc điều khiển.
- Hiểu sâu về con trỏ (pointers), địa chỉ bộ nhớ và tham chiếu (references).
- Phân biệt rõ ràng giữa cấp phát bộ nhớ tĩnh (Stack) và cấp phát bộ nhớ động (Heap).
- Học cách quản lý bộ nhớ thủ công bằng `new` và `delete`, và cách tránh rò rỉ bộ nhớ (memory leaks).
- (Nâng cao) Làm quen với smart pointers trong C++ hiện đại (C++11 trở lên).
- Nhận thức được các rủi ro bảo mật liên quan đến quản lý bộ nhớ như buffer overflow, use-after-free.

English

- Master the basic syntax of C++: variables, data types, and control structures.
- Gain a deep understanding of pointers, memory addresses, and references.
- Clearly distinguish between static memory allocation (Stack) and dynamic memory allocation (Heap).
- Learn manual memory management using `new` and `delete`, and how to avoid memory leaks.
- (Advanced) Get introduced to smart pointers in modern C++ (C++11 and later).
- Become aware of memory-related security risks such as buffer overflows and use-after-free vulnerabilities.

Linh Kiện & Dụng Cụ / Components & Tools

Tiếng Việt (Vietnamese)

- Máy tính cá nhân (Windows, macOS hoặc Linux).
- Trình biên dịch C++: GCC (MinGW trên Windows, mặc định trên Linux) hoặc Clang (trên macOS).
- Môi trường phát triển tích hợp (IDE): Visual Studio Code, CLion, hoặc Code::Blocks.
- Tiện ích mở rộng C/C++ cho VS Code (nếu sử dụng VS Code).

- Công cụ gỡ lỗi (Debugger): GDB hoặc LLDB.

English

- Personal Computer (Windows, macOS, or Linux).
- C++ Compiler: GCC (MinGW on Windows, default on Linux) or Clang (on macOS).
- Integrated Development Environment (IDE): Visual Studio Code, CLion, or Code::Blocks.
- C/C++ extension for VS Code (if using VS Code).
- Debugger: GDB or LLDB.

Lý Thuyết / Theory

1. Giới thiệu C++ Cơ bản / Introduction to C++ Basics

Tiếng Việt

C++ là một ngôn ngữ lập trình mạnh mẽ, hỗ trợ lập trình hướng đối tượng, lập trình thủ tục và lập trình generic. Nó là phần mở rộng của ngôn ngữ C và được sử dụng rộng rãi trong lập trình hệ thống, phát triển game, phần mềm hiệu năng cao và đặc biệt là trong các hệ thống nhúng và bảo mật mạng (Cybersecurity) vì nó cho phép can thiệp sâu vào bộ nhớ.

Một chương trình C++ cơ bản luôn có hàm `main()`, nơi bắt đầu thực thi chương trình.

English

C++ is a powerful programming language that supports object-oriented, procedural, and generic programming. It is an extension of the C language and is widely used in systems programming, game development, high-performance software, and especially in embedded systems and Cybersecurity because it allows low-level memory manipulation.

A basic C++ program always has a `main()` function, where execution begins.

2. Biến và Kiểu dữ liệu / Variables and Data Types

Tiếng Việt

Các kiểu dữ liệu cơ bản bao gồm:

- `int`: Số nguyên (thường là 4 bytes).
- `char`: Ký tự (1 byte).
- `float`, `double`: Số thực dấu phẩy động.
- `bool`: Kiểu logic (true/false).

Biến phải được khai báo với kiểu dữ liệu trước khi sử dụng.

English

Basic data types include:

- **int**: Integer (usually 4 bytes).
- **char**: Character (1 byte).
- **float**, **double**: Floating-point numbers.
- **bool**: Boolean type (true/false).

Variables must be declared with a data type before they are used.

3. Con Trỏ và Địa chỉ Bộ nhớ / Pointers and Memory Addresses

Tiếng Việt

Mỗi biến trong chương trình được lưu trữ tại một vị trí cụ thể trong bộ nhớ RAM, gọi là **địa chỉ** (address).

Toán tử **&** (address-of operator) được sử dụng để lấy địa chỉ của một biến.

Con trỏ (Pointer) là một biến đặc biệt dùng để lưu trữ địa chỉ của một biến khác.

Khai báo con trỏ sử dụng ký hiệu *****.

Toán tử ***** (dereference operator) khi dùng trước một con trỏ sẽ truy cập vào giá trị đang được lưu tại địa chỉ mà con trỏ đang trỏ tới.

English

Every variable in a program is stored at a specific location in RAM, called an **address**.

The **&** (address-of) operator is used to get the memory address of a variable.

A **Pointer** is a special variable used to store the memory address of another variable.

Pointers are declared using the ***** symbol.

The ***** (dereference) operator, when used before a pointer, accesses the value stored at the address the pointer is pointing to.

4. Phân biệt Stack và Heap / Distinguishing Stack and Heap

Tiếng Việt

Bộ nhớ cấp phát cho một chương trình C++ được chia thành nhiều phần, quan trọng nhất là **Stack** và **Heap**.

Stack (Ngăn xếp):

- Được quản lý tự động bởi CPU.
- Các biến địa phương (local variables) được cấp phát trên Stack khi hàm được gọi và tự động giải phóng khi hàm kết thúc.
- Tốc độ truy cập rất nhanh.
- Kích thước giới hạn (thường vài Megabytes). Vượt quá giới hạn sẽ gây ra lỗi **Stack Overflow**.
- Cấu trúc dữ liệu theo nguyên tắc LIFO (Last In, First Out).

Heap (Vùng nhớ động):

- Được quản lý thủ công bởi lập trình viên.
- Dùng cho cấp phát bộ nhớ động (Dynamic memory allocation) trong lúc chạy chương trình (runtime).
- Kích thước lớn, bị giới hạn bởi bộ nhớ RAM của hệ thống.
- Tốc độ truy cập chậm hơn Stack do phải dùng con trỏ để truy xuất.
- Nếu không giải phóng bộ nhớ sau khi dùng xong, sẽ xảy ra hiện tượng **Rò rỉ bộ nhớ (Memory Leak)**.

English

The memory allocated to a C++ program is divided into several segments, the most important being the **Stack** and the **Heap**.

Stack:

- Managed automatically by the CPU.
- Local variables are allocated on the Stack when a function is called and automatically deallocated when the function exits.
- Very fast access speed.
- Limited size (usually a few Megabytes). Exceeding this limit causes a **Stack Overflow**.
- Data structure follows the LIFO (Last In, First Out) principle.

Heap (Dynamic Memory):

- Managed manually by the programmer.
- Used for dynamic memory allocation during runtime.
- Large size, limited by the system's available RAM.
- Access speed is slower than Stack because pointers must be used to access data.
- Failure to free memory after use results in a **Memory Leak**.

5. Quản lý Bộ nhớ Động / Dynamic Memory Management

Tiếng Việt

Trong C++, cấp phát động được thực hiện thông qua toán tử **new**.
Giải phóng bộ nhớ động được thực hiện thông qua toán tử **delete**.

Ví dụ:

```
int* p = new int; // Cpp phát 1 s nguyên trên Heap *p = 10; delete p;  
// Giải phóng bộ nhớ p = nullptr; // Tránh con trỏ lơ lửng (Dangling pointer)
```

Với mảng:

```
int* arr = new int[100]; // Cpp phát mảng 100 phần tử trên Heap delete[]  
arr; // Giải phóng mảng (chú ý có dấu ngoặc vuông [])
```

English

In C++, dynamic allocation is performed using the `new` operator. Dynamic memory deallocation is performed using the `delete` operator.

Example:

```
int* p = new int; // Allocate 1 integer on the Heap *p = 10; delete p; //  
Free memory p = nullptr; // Avoid dangling pointer
```

For arrays:

```
int* arr = new int[100]; // Allocate an array of 100 integers on the Heap  
delete[] arr; // Free the array (note the square brackets [])
```

6. *Rủi ro Bảo mật: Buffer Overflow & Dangling Pointers / Security Risks: Buffer Overflow & Dangling Pointers*

Tiếng Việt

- **Buffer Overflow (Tràn bộ đệm):** Xảy ra khi chương trình ghi dữ liệu vượt quá kích thước được cấp phát cho một khối nhớ (như mảng). Hacker có thể lợi dụng điều này để ghi đè lên bộ nhớ lân cận, thay đổi luồng thực thi của chương trình.
- **Dangling Pointer (Con trỏ lơ lửng):** Khi bạn `delete` một con trỏ nhưng không gán nó bằng `nullptr`, con trỏ đó vẫn giữ địa chỉ bộ nhớ cũ (đã được trả lại cho hệ điều hành). Truy cập vào địa chỉ này sẽ gây lỗi hoặc tạo lỗ hổng **Use-After-Free (UAF)**, một dạng khai thác bảo mật nghiêm trọng.

English

- **Buffer Overflow:** Occurs when a program writes data beyond the size allocated for a memory block (like an array). Hackers can exploit this to overwrite adjacent memory, altering the program's execution flow.
- **Dangling Pointer:** When you `delete` a pointer but do not set it to `nullptr`, the pointer still holds the old memory address (which has been returned to the OS). Accessing this address causes errors or creates a **Use-After-Free (UAF)** vulnerability, a severe security exploit.

Sơ Đồ Cấu Hình Mạng / Network Topology

Tiếng Việt

Bài học này tập trung vào lập trình ứng dụng nội bộ (local application) xử lý bộ nhớ, không yêu cầu thiết lập mạng. Tuy nhiên, các kỹ thuật phân tích bộ nhớ này là nền tảng để phát hiện lỗ hổng trên các dịch vụ mạng (Network Services).

English

This lesson focuses on local application programming and memory handling, requiring no network setup. However, these memory analysis techniques are foundational for discovering vulnerabilities in Network Services.

Thực Hành / Hands-On

Bài 1: Hiểu về Địa chỉ và Giá trị (Understanding Addresses and Values)

Yêu cầu (Requirements)

Viết chương trình tạo một biến số nguyên, in ra giá trị và địa chỉ của biến đó. Sau đó dùng con trỏ để thay đổi giá trị.

Các bước (Steps)

1. Khai báo biến `int a = 10;`
2. In ra `a` và `&a`.
3. Khai báo con trỏ `int* ptr = &a;`
4. Dùng `*ptr = 20;` để thay đổi giá trị.
5. In lại `a`.

Bài 2: Khám phá Stack (Exploring the Stack)

Yêu cầu (Requirements)

Chứng minh rằng các biến địa phương được cấp phát trên Stack và hiểu về giới hạn của Stack.

Các bước (Steps)

1. Khai báo liên tiếp các biến `int x, y, z;`. In địa chỉ của chúng ra. (Bạn sẽ thấy chúng nằm sát nhau trong bộ nhớ).
2. Tạo một hàm đệ quy vô hạn mà không có điều kiện dừng.
3. Chạy chương trình và quan sát lỗi **Stack Overflow** (Segmentation fault).

Bài 3: Cấp phát Động với Heap (Dynamic Allocation with Heap)

Yêu cầu (Requirements)

Tạo mảng động trên Heap dựa trên kích thước do người dùng nhập.

Các bước (Steps)

1. Hỏi người dùng kích thước `n`.
2. Dùng `int* arr = new int[n];`.
3. Nhập dữ liệu và in dữ liệu mảng.
4. Xóa mảng bằng `delete[] arr;`
5. Gán `arr = nullptr;`

Bài 4: Trải nghiệm Rò rỉ Bộ nhớ (Experiencing Memory Leaks) (LÀM CẦN THẬN)

Yêu cầu (Requirements)

Tạo một vòng lặp liên tục cấp phát bộ nhớ nhưng không giải phóng để xem chương trình tiêu tốn RAM.

Các bước (Steps)

1. Tạo vòng lặp `while(true)`.
2. Bên trong, `long* leaky = new long[1000000];`
3. Mở Task Manager (Windows) hoặc Activity Monitor (Mac) để theo dõi lượng RAM bị ngốn.
4. Ngừng chương trình trước khi máy tính bị đơ!

Cảnh Báo An Toàn & Đạo Đức / Safety Warnings and Ethical Notices

Tiếng Việt

- **Chú ý hệ thống:** Khi thực hành bài 4 (Memory Leak) hoặc bài 2 (Stack Overflow), chương trình sẽ bị crash. Đảm bảo bạn đã lưu mọi dữ liệu quan trọng trước khi chạy mã cố ý gây lỗi hệ thống.
- **Mục đích giáo dục:** Các lỗ hổng như Buffer Overflow và Use-After-Free được dạy để HIỂU và PHÒNG CHỐNG. TUYỆT ĐỐI KHÔNG sử dụng các kỹ thuật này để khai thác các hệ thống không thuộc quyền sở hữu của bạn.
- Viết mã an toàn (Secure Coding) là trách nhiệm của một lập trình viên chuyên nghiệp. Luôn luôn kiểm tra ranh giới mảng (array bounds checking) và quản lý chặt chẽ vòng đời của con trỏ.

English

- **System Caution:** When practicing Exercise 4 (Memory Leak) or Exercise 2 (Stack Overflow), the program will crash. Ensure you have saved all important data before running code that intentionally causes system errors.
 - **Educational Purpose:** Vulnerabilities like Buffer Overflows and Use-After-Free are taught to UNDERSTAND and PREVENT them. ABSOLUTELY DO NOT use these techniques to exploit systems you do not own.
 - Secure Coding is the responsibility of a professional programmer. Always perform array bounds checking and strictly manage the lifecycle of pointers.
-

Code Mẫu / Code Samples

Sample 1: Pointers Basics (Con trỏ cơ bản)

```
#include <iostream> using namespace std; int main() { int value = 42;
int* ptr = &value; // ptr lưu địa chỉ của value cout << "Gia tri cua
value: " << value << endl; cout << "Dia chi cua value (&value): " <<
&value << endl; cout << "Gia tri cua ptr (dia chi ma no luu): " << ptr <<
endl; cout << "Gia tri ma ptr tro toi (*ptr): " << *ptr << endl; // Thay
đổi giá trị thông qua con trỏ *ptr = 100; cout << "\nSau khi thay doi
*ptr = 100:" << endl; cout << "Gia tri moi cua value: " << value << endl;
return 0; }
```

Sample 2: Stack vs Heap Allocation (Cấp phát tĩnh và động)

```
#include <iostream> using namespace std; void createOnStack() { int
stackVar = 10; // Biến này nằm trên Stack cout << "Gia tri Stack: " <<
stackVar << ", Dia chi: " << &stackVar << endl; // Biến này sẽ bị xóa
mất khi hàm này kết thúc } void createOnHeap() { int* heapVar = new int;
// Cấp phát trên Heap *heapVar = 20; cout << "Gia tri Heap: " << *heapVar
<< ", Dia chi: " << heapVar << endl; // PHẢI gọi phóng thủ công delete
heapVar; heapVar = nullptr; // Thực hành tốt: Tránh Dangling pointer }
int main() { createOnStack(); createOnHeap(); return 0; }
```

Sample 3: Dynamic Array Management (Quản lý mảng động)

```
#include <iostream> using namespace std; int main() { int n; cout <<
"Nhap so luong phan tu: "; cin >> n; // Cấp phát mảng động trên Heap
int* arr = new int[n]; // Kiểm tra xem bộ nhớ có được cấp phát thành
công không if (arr == nullptr) { cerr << "Loi cap phat bo nho!" << endl;
return 1; } // Nhập liệu for(int i = 0; i < n; i++) { arr[i] = i * 2; }
// Xuất liệu cout << "Cac phan tu trong mang: "; for(int i = 0; i < n;
i++) { cout << arr[i] << " "; } cout << endl; // Gọi phóng bộ nhớ cho
MẢNG delete[] arr; arr = nullptr; return 0; }
```

Sample 4: Dangling Pointer Vulnerability (Lỗi hỏng con trỏ lơ lửng - Use After Free)

```
#include <iostream> using namespace std; int main() { int* ptr = new
int(1337); cout << "Gia tri ban dau: " << *ptr << endl; delete ptr; // Bộ
nhớ đã được trả lại cho hệ thống // LỖI BỔ MẤT: ptr bây giờ là
dangling pointer // Hacker có thể kiểm soát vùng nhớ này // Caso tình truy
cập vùng nhớ đã bị xóa (Use After Free) cout << "Gia tri sau khi xoa
(Undefined behavior): " << *ptr << endl; // Cách phòng tránh: // ptr =
nullptr; return 0; }
```

Câu Hỏi Thảo Luận / Discussion

Tiếng Việt

1. Tại sao không nên lưu trữ toàn bộ dữ liệu trên Stack?

2. Điều gì sẽ xảy ra nếu bạn cấp phát bộ nhớ trên Heap trong một vòng lặp vô hạn mà quên `delete`? Trình điều hành sẽ phản ứng như thế nào?
3. Tại sao lỗi hỏng Use-After-Free lại nguy hiểm? Hacker có thể làm gì nếu họ có thể thao tác với vùng nhớ đã bị `delete`?
4. Khác biệt giữa `delete ptr` và `delete[] ptr` là gì? Điều gì xảy ra nếu dùng sai?

English

1. Why shouldn't we store all data on the Stack?
2. What happens if you allocate memory on the Heap in an infinite loop and forget to `delete` it? How will the Operating System react?
3. Why is the Use-After-Free vulnerability dangerous? What could a hacker do if they manipulate memory that has already been `deleted`?
4. What is the difference between `delete ptr` and `delete[] ptr`? What happens if you use the wrong one?

Bài Về Nhà / Homework

Tiếng Việt

Nhiệm vụ 1: Viết một chương trình quản lý sinh viên đơn giản (tên, điểm) sử dụng một mảng cấp phát động. Cho phép người dùng nhập số lượng sinh viên, nhập thông tin, in danh sách, và cuối cùng giải phóng bộ nhớ đúng cách.

Nhiệm vụ 2: Viết một hàm nhận vào 2 con trỏ `int* a` và `int* b`. Hàm này sẽ hoán đổi (swap) giá trị của 2 biến mà các con trỏ này trỏ tới. Gọi hàm này trong `main` để kiểm chứng.

Nhiệm vụ 3 (Nghiên cứu): Tìm hiểu về `std::unique_ptr` và `std::shared_ptr` trong C++11 (Smart Pointers). Viết lại "Nhiệm vụ 1" nhưng sử dụng smart pointers thay vì `new` và `delete` thủ công.

English

Task 1: Write a simple student management program (name, grade) using a dynamically allocated array. Allow the user to input the number of students, input data, print the list, and finally properly deallocate the memory.

Task 2: Write a function that takes two pointers `int* a` and `int* b`. This function should swap the values of the variables these pointers point to. Call this function in `main` to verify.

Task 3 (Research): Research `std::unique_ptr` and `std::shared_ptr` in C++11 (Smart Pointers). Rewrite "Task 1" but use smart pointers instead of manual `new` and `delete`.

Đánh Giá / Assessment Rubric

Tiêu chí / Criteria	Xuất sắc / Excellent (9-10)	Tốt / Good (7-8)	Đạt / Pass (5-6)	Cần cố gắng / Needs Work (<5)
Hiểu lý thuyết (Theory Understanding)	Giải thích chính xác và phân biệt rạch ròi Stack/Heap, con trỏ. (Explains and distinguishes Stack/Heap and pointers perfectly).	Hiểu cơ bản nhưng còn nhầm lẫn nhỏ ở khái niệm khó. (Basic understanding but minor confusion on hard concepts).	Nắm được định nghĩa nhưng chưa giải thích được sự khác biệt sâu sắc. (Knows definitions but can't explain deep differences).	Không phân biệt được Stack và Heap. (Cannot distinguish Stack and Heap).
Thực hành (Hands-On)	Hoàn thành tất cả các bài tập, code chạy mượt mà, không có memory leak. (Completes all labs, code runs smoothly, zero memory leaks).	Hoàn thành bài tập, có một vài lỗi nhỏ dễ sửa. (Completes labs, some minor fixable bugs).	Code chạy được nhưng còn dư thừa, cấu trúc chưa rõ ràng. (Code runs but is messy, unclear structure).	Code không biên dịch được hoặc crash liên tục. (Code fails to compile or crashes constantly).
Quản lý Bộ nhớ (Memory Management)	Luôn dùng <code>delete/delete[]</code> đúng cặp với <code>new/new[]</code> , gán <code>nullptr</code> sau khi xóa. (Always pairs <code>new/delete</code> correctly, uses <code>nullptr</code>).	Đa số giải phóng bộ nhớ đúng, quên gán <code>nullptr</code> . (Mostly frees memory correctly, forgets <code>nullptr</code>).	Có rò rỉ bộ nhớ ở một vài trường hợp ngoại lệ. (Has memory leaks in edge cases).	Quên hoàn toàn việc giải phóng bộ nhớ (No memory deallocation).

Bảo mật & Đạo đức (Security & Ethics)	Hiểu và phòng tránh tốt Buffer Overflow & Use-After-Free. (Understands and prevents BO & UAF well).	Có nhận thức về bảo mật nhưng chưa áp dụng chặt chẽ vào code. (Aware of security but loosely applied to code).	Nhận thức hạn chế về rủi ro bộ nhớ. (Limited awareness of memory risks).	Viết code không an toàn, dễ bị khai thác. (Writes unsafe, exploitable code).
--	---	--	--	--

(Bản quyền khóa học / Course Copyright: Aero-Fullstack4kid - CyberSec AI 10 Weeks)
(End of Week 3)

Phụ lục: Các kỹ thuật nâng cao với con trỏ (Appendix: Advanced Pointer Techniques)

Tiếng Việt

Trong môi trường thực tế, đặc biệt là lập trình hệ thống (System Programming) hay thiết kế hệ điều hành, con trỏ còn được sử dụng cho nhiều mục đích phức tạp hơn:

1. **Con trỏ hàm (Function Pointers):** Con trỏ không chỉ trỏ đến dữ liệu (biến, mảng) mà còn có thể trỏ đến một hàm. Điều này cho phép truyền một hàm như một tham số (callback function), được dùng rất nhiều trong C và C++ cũ.
2. **Con trỏ Void (Void Pointers):** `void*` là con trỏ có thể trỏ đến bất kỳ kiểu dữ liệu nào. Nó rất hữu ích khi bạn không biết trước kiểu dữ liệu sẽ được truyền vào (generic programming trong C). Tuy nhiên, trước khi sử dụng giá trị, bạn bắt buộc phải ép kiểu (typecast) nó về kiểu dữ liệu chính xác. Việc sử dụng `void*` có thể gây mất an toàn kiểu dữ liệu (type safety) nên cần rất thận trọng.
3. **Mảng con trỏ (Array of Pointers):** Thay vì lưu trữ các đối tượng trực tiếp, bạn có thể lưu các con trỏ trỏ tới các đối tượng. Rất hữu ích để lưu một mảng các chuỗi (mảng char) có độ dài khác nhau mà không tốn bộ nhớ dư thừa.
4. **Con trỏ tới con trỏ (Pointer to Pointer):** Ký hiệu là `**`. Được dùng phổ biến khi làm việc với mảng hai chiều động (dynamic 2D arrays).

English

In a real-world environment, especially in System Programming or operating system design, pointers are used for much more complex purposes:

1. **Function Pointers:** A pointer can point not only to data (variables, arrays) but also to a function. This allows passing a function as a parameter (callback function), which is heavily used in C and older C++ codebases.
2. **Void Pointers:** `void*` is a pointer that can point to any data type. It is very useful when you do not know in advance what type of data will be passed (generic programming in C). However, before using the value, you must typecast it back to the

exact data type. Using `void*` can bypass type safety, so it must be used with extreme caution.

3. Array of Pointers: Instead of storing objects directly, you can store pointers that point to those objects. Very useful for storing an array of strings (char arrays) of varying lengths without wasting memory.

4. Pointer to Pointer: Denoted by `**`. Commonly used when working with dynamic two-dimensional arrays (dynamic 2D arrays).

Phụ lục 2: Thực hành Mảng 2 chiều Động (Appendix 2: Dynamic 2D Array Practice)

Tiếng Việt

Cách tạo một mảng 2 chiều động kích thước hàng (rows) x cột (cols):

```
int rows = 3, cols = 4; // 1. Cpp phát mảng con trỏ lưu trữ địa chỉ của các hàng int** matrix = new int*[rows]; // 2. Cpp phát bộ nhớ cho từng hàng for(int i = 0; i < rows; i++) { matrix[i] = new int[cols]; } // 3. Sử dụng matrix[i][j] bình thường // 4. Giải phóng bộ nhớ (Phải làm ngược lại quá trình cpp phát) for(int i = 0; i < rows; i++) { delete[] matrix[i]; // Giải phóng từng hàng trước } delete[] matrix; // Giải phóng mảng con trỏ matrix = nullptr;
```

English

How to create a dynamic 2D array of size rows x cols:

```
int rows = 3, cols = 4; // 1. Allocate an array of pointers to hold row addresses int** matrix = new int*[rows]; // 2. Allocate memory for each row for(int i = 0; i < rows; i++) { matrix[i] = new int[cols]; } // 3. Use matrix[i][j] normally // 4. Deallocate memory (Must be done in reverse order of allocation) for(int i = 0; i < rows; i++) { delete[] matrix[i]; // Free each row first } delete[] matrix; // Free the array of pointers matrix = nullptr;
```

(Cuối tài liệu thực sự / Actual end of document)